The Journal of Open Source Software

GCol: A High-Performance Python Library for Graph Colouring

Rhyd Lewis [1¶] **and Geraint Palmer** [1]

1 School of Mathematics, Cardiff University, Wales, United Kingdom ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Vissarion Fisikopoulos

Reviewers:

- @paulbrodersen
- @IvanIsCoding

Submitted: 19 February 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Graph colouring is the computational problem of assigning colours to entities of a graph so that adjacent entities receive different colours. The aim is to use as few colours as possible. In general terms, a graph is a mathematical object comprising a set of *nodes* and a set of *edges* that join pairs of nodes. Graphs are also known as *networks*, nodes as *vertices*, and edges as *links*. Examples of graph colouring are shown in Figure 1, which help to demonstrate the following principles.

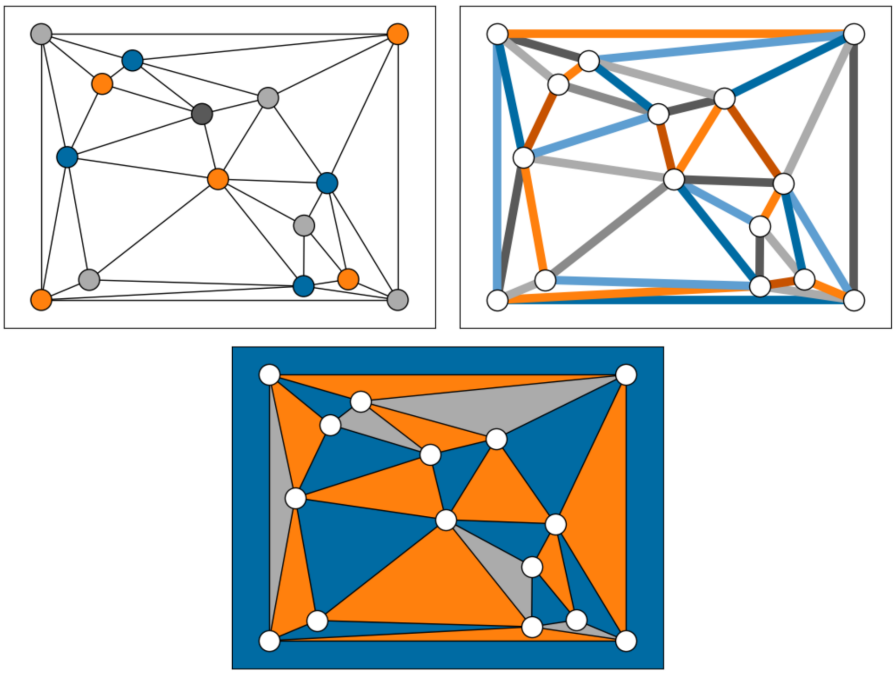

Figure 1: A node colouring, edge colouring, and face colouring (respectively), of a fifteen-node planar graph. Each of these examples uses the minimum number of colours and was generated using GCol's colouring and visualisation routines.

- A *node colouring* is an assignment of colours to the nodes of a graph so that adjacent nodes have different colours. The smallest number of colours needed for the nodes of a graph G is known as its chromatic number, denoted by $\chi(G)$. Identification of $\chi(G)$ is an NP-hard problem.
- An *edge colouring* is an assignment of colours to the edges of a graph so that all adjacent

edges have different colours. The smallest number of colours needed for the edges of a graph G is known at the chromatic index, denoted by $\chi'(G)$. According to Vizing's theorem (*Vizing's Theorem*, 2025), $\chi'(G)$ is either $\Delta(G)$ or $\Delta(G)+1$, where $\Delta(G)$ is the maximum degree in G. Identifying $\chi'(G)$ is also NP-hard.

- A *face colouring* is an assignment of colours to the faces of a planar embedding so that all adjacent faces have different colours. Note, that planar embeddings only exist for planar graphs. The smallest number of colours needed to colour the faces of a planar embedding is known as its face chromatic number. Due to the Four Colour Theorem, this number never exceeds four and, unlike $\chi(G)$ and $\chi'(G)$, can be determined in polynomial time (Robertson et al., 1997).

Graph colouring has applications in many practical areas including timetabling, sports league scheduling, designing seating plans, code optimisation, and solving Sudoku puzzles (Lewis, 2021b). It is also a topic of great theoretical interest (Cranston, 2024) that often appears in university-level courses on graph theory, algorithms, and combinatorics.

GCol is a new, open-source Python library for graph colouring built on top of the well-known NetworkX library (Hagberg et al., 2024). It provides easy-to-use, high-performance algorithms for the above three problems, as well as routines for equitable colouring, weighted colouring, pre-colouring, maximum independent set identification, and solution visualisation. The following code snippet shows how to use the library to create a dodecahedral graph G, colour its nodes using $\chi(G) = 3$ colours, and then output the solution in textual and diagrammatic form (see Figure 2):

```
>>> import networkx as nx
>>> import matplotlib.pyplot as plt
>>> import gcol
>>> G = nx.dodecahedral_graph()
>>> c = gcol.node_coloring(G)
>>> print("Node coloring of G =", c)
Node coloring of G = {0: 0, 1: 1, 19: 1, 10: 1, 2: 0, 3: 2, 8: 0, 9: 2, 18: 0,
11: 2, 6: 1, 7: 2, 4: 0, 5: 2, 13: 0, 12: 1, 14: 1, 15: 0, 16: 2, 17: 1}
>>> nx.draw_networkx(G, node_color=gcol.get_node_colors(G, c))
>>> plt.show() # See Figure 2
```

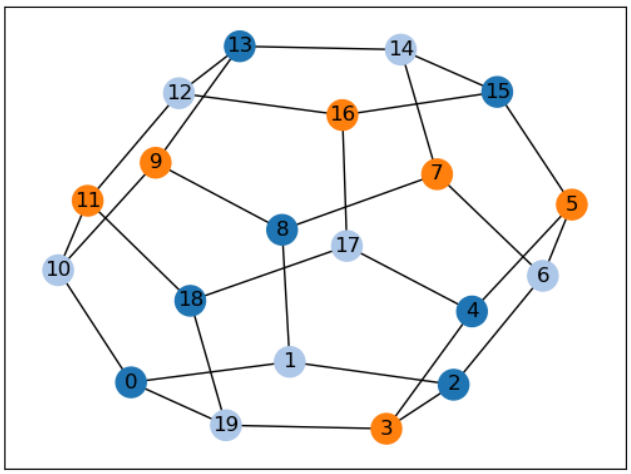

Figure 2: Output from the code snippet.

DRAFT

Statement of need

Open-source resources for graph colouring have existed for some time, primarily for node colouring. An early example is the PL/I code for node colouring included in the 1979 paper of (Leighton, 1979). Stand-alone C-based resources were also made available online in the mid-1990s Trick (1994). A Java-based package implementing the methods of (Culberson, 1994) is now also available (Shah, 2020).

Graph colouring functionality is also included in some popular open-source C++ libraries. (*Boost Graph Library*, 2025) uses a simple constructive heuristic for node colouring, while (*Lemon Graph Library*, 2025) includes a method for colouring the nodes of planar graphs using at most five colours. (*Goblin Graph Library*, 2025), features a similar method to Lemon and, in addition, includes a mixed integer linear programming approach for exactly solving the node and edge colouring problems. This algorithm has an exponential time complexity and, consequently, is unsuitable for larger problem instances.

An option that also includes visualisation tools is provided by (*SageMath*, 2025). This has methods for both node and edge colouring, and can also enumerate *all* node colourings of a graph. Like Goblin, however, its algorithms are based on integer programming and operate in exponential time. NetworkX itself also includes some simple greedy heuristics for node colouring. In addition, it features an exact polynomial-time algorithm for balancing the number of nodes per colour (equitable node colouring); however, this can only be applied when the number of available colours exceeds $\Delta(G)$—for fewer colours, the problem is NP-hard and no functionality is available.

Further specialised methods for node colouring are also provided by (*ColPack*, 2025) and in the algorithm suite of (Lewis, 2021a), both in C++. The algorithms of ColPack are "greedy heuristics in the sense that the algorithms progressively extend a partial colouring by processing one vertex at a time, in some order, in each step assigning a vertex the smallest allowable colour" (*ColPack*, 2025). In contrast, the suite of (Lewis, 2021a) features several contrasting algorithms, including constructive heuristics, an exact algorithm based on backtracking, and bespoke metaheuristics.

The above survey suggests that existing open-source options for graph colouring are limited. Current resources tend to either use simple constructive heuristics that lead to low-quality solutions, or exponential-time exact algorithms that cannot cope with larger graphs. There are also few options for edge colouring, equitable colouring, and solution visualisation, and, to our knowledge, no options for face colouring, weighted colouring, or pre-colouring.

The GCol library features routines for all of the above. Optimisation is performed by a choice of node-colouring algorithms that include an exact, heuristically-guided, exponential-time backtracking algorithm, and several high-performance polynomial-time heuristics. The latter methods combine fast constructive methods with contemporary local search heuristics that extend the C++ implementations of (Lewis, 2021a), allowing high-quality solutions to be generated in reasonable run times, even for very large graphs. Edge colourings and face colourings are also determined by these algorithms by colouring, respectively, the nodes of the corresponding line graphs and dual graphs.

Performance

The various optimisation algorithms available in the GCol library are described in detail in its official documentation (*GCol*, 2025) and in the book of (Lewis, 2021b). These resources include detailed information on the asymptotic complexity of all algorithms used.

DRAFT

The Journal of Open Source Software

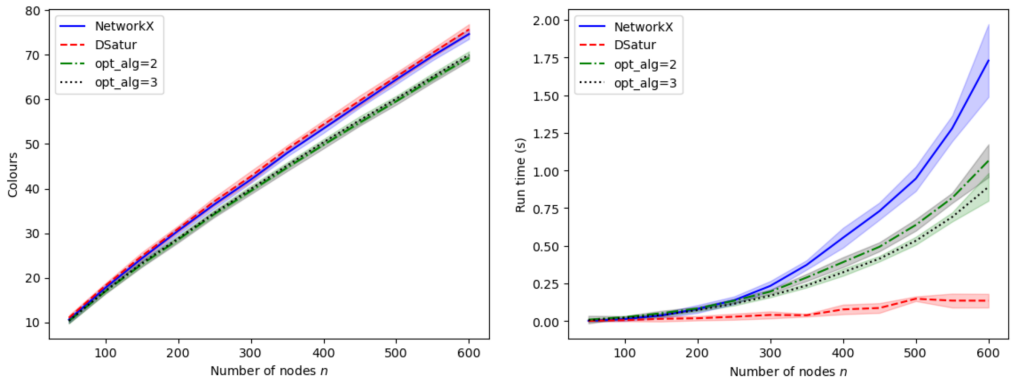

Figure 3: Number of colours (left) and run times (right) of different node colouring algorithms with $G(n, 0.5)$ graphs. Each point in the charts is a mean across fifty graphs. Shaded areas indicate one standard deviation on either side of these means. All results were found by executing the code on a 3.0 GHtz Windows 11 PC with 16 GB of RAM.

To give an indication of performance, Figure 3 compares two of GCol's local search routines (`opt_alg=2` and `opt_alg=3`) to NetworkX's "interchange" operator and the well-known DSatur heuristic (*DSatur*, 2025). For comparative purposes, both local search algorithms use a fixed iteration limit equal to the number of nodes n in the graph, though the user is free to use different values. Trials were conducted on randomly generated Erdos-Renyi graphs, commonly denoted by $G(n, p)$. These graphs are constructed by taking n nodes and adding an edge between each node pair at random with probability p. The expected number of edges in a $G(n, p)$ graph is therefore $\binom{n}{2}p$ and expected node degrees are $p(n-1)$. For these tests, we use $p = 0.5$ and differing values for n. This is due to the result of (Wormald, 1987), who has established that for $n \gtrapprox 30$, a set of randomly constructed $G(n, 0.5)$ graphs can be considered equivalent to a random sample from the population of all unlabelled n-node graphs. This allows us to make general statistical statements about the performance of an algorithm across the set of all n-node graphs, though different observations are likely when executing these algorithms on particular topologies.

Figure 3 clearly shows that GCol's local search algorithms make significant improvements to the solutions provided by the DSatur strategy, albeit with additional time requirements. The solutions and run times of the local search algorithms are also superior to NetworkX's node colouring routines. Further improvements in solution quality will also be achieved by increasing the iteration limit of the local search algorithms.

Results concerning the performance of GCol's exact algorithms, equitable colouring functionality, and other associated optimisation problems are reported as part of GCol's documentation, (*GCol*, 2025).

References

Boost graph library. (2025). https://www.boost.org/doc/libs/1_46_1/libs/graph/doc/index.html.

ColPack. (2025). https://cscapes.cs.purdue.edu/coloringpage/software.htm.

Cranston, D. (2024). *Guide coloring methods*. Cranston, Richmond, Virginia. ISBN: 979-8-218-46240-0

Culberson, J. (1994). *Culberson's node coloring implementations*. https://www3.cs.stonybrook.edu/~algorith/implement/culberson/distrib/.

DSatur. (2025). https://en.wikipedia.org/wiki/DSatur.

DRAFT

GCol: A library for graph coloring. (2025). https://gcol.readthedocs.io/en/latest/.

Goblin graph library. (2025). https://goblin2.sourceforge.net/.

Hagberg, A., Schult, D., & Swart, P. (2024). *NetworkX: Network analysis in Python*. https://networkx.org/.

Leighton, F. T. (1979). A graph coloring algorithm for large scheduling problems. *Journal of Research of the National Bureau of Standards*, *84*(6), 489–506.

Lemon graph library. (2025). https://lemon.cs.elte.hu/trac/lemon.

Lewis, R. (2021a). *A guide to graph colouring: User guide*. https://rhydlewis.eu/gcol/.

Lewis, R. (2021b). *Guide to graph colouring: Algorithms and applications*. Springer Cham. ISBN: 978-3-030-81056-6

Robertson, N., Sanders, D., Seymour, P., & Thomas, R. (1997). The four color theorem. *Journal of Combinatorial Theory*, *70*, 2–44.

SageMath: Graph coloring. (2025). https://doc.sagemath.org/html/en/reference/graphs/sage/graphs/graph_coloring.html.

Shah, S. (2020). JCOL: A java package for solving the graph coloring problem. *Journal of Open Source Software*, *5*(48), 1843. https://doi.org/10.21105/joss.01843

Trick, M. (1994). *COLOR.C: Easy code for graph coloring*. https://www3.cs.stonybrook.edu/~algorith/implement/culberson/distrib/.

Vizing's theorem. (2025). https://en.wikipedia.org/wiki/Vizing%27s_theorem.

Wormald, N. (1987). Generating random unlabelled graphs. *SIAM Journal on Computing*, *16*(4), 717–727. https://doi.org/10.1137/0216048

DRAFT